

Aula 2 — Banco de dados, ActiveRecord e o model `User`

Duração: ~3h · **Você sai daqui com:** um `User` com validações e senha hashada, testado.

Checkpoint: `git checkout aula-02`

1. Por que Postgres, e não SQLite

SQLite é um arquivo. Funciona muito bem para um app de celular ou um script. Para um servidor com várias requisições ao mesmo tempo, ele trava — só uma escrita por vez no banco inteiro.

Postgres é um servidor: aguenta concorrência, tem tipos ricos (`jsonb`, arrays, intervalos de tempo), índices parciais e transações de verdade. É o que o `seem-backend` usa em produção, então é o que usamos aqui — **desenvolver no mesmo banco da produção evita a categoria inteira de bug que só aparece no deploy.**

Transação

Um bloco de operações que acontece **inteiro ou não acontece**. Deu erro no meio, o banco desfaz tudo — isso se chama *rollback*.

O exemplo clássico é transferir dinheiro: debitar de um e creditar no outro têm que ser a mesma operação. Debitar sozinho é dinheiro que sumiu.

```
User.transaction do
  user.save!
  user.invalidate_sessions!
  user.clear_password_reset_code!
end
```

Repare no `!`: dentro de uma transação você **quer** que estoure. `save` devolve `false` em silêncio e a transação seguiria feliz, gravando metade.

Na Aula 3, trocar a senha vai precisar exatamente disso.

2. ActiveRecord

É o ORM do Rails: cada classe é uma tabela, cada objeto é uma linha.

Django ORM	ActiveRecord
<code>User.objects.filter(role="admin")</code>	<code>User.where(role: "admin")</code>
<code>User.objects.get(id=1)</code>	<code>User.find(1)</code>
<code>User.objects.all().order_by("-created_at")</code>	<code>User.order(created_at: :desc)</code>
<code>User.objects.count()</code>	<code>User.count</code>
<code>u.save()</code>	<code>u.save</code>

A diferença de filosofia: no Django você declara os campos na classe. **No Rails a classe não declara nada** — ela lê as colunas do banco em tempo de execução. A fonte da verdade é a migration.

```
class User < ApplicationRecord
end
```

Isso já tem `name`, `email`, `id`, `created_at` — tudo que existir na tabela `users`.

ActiveSupport: os métodos que o Rails inventou

O Rails adiciona métodos às classes do próprio Ruby. Isso é a gem `activesupport`, não Ruby puro — fora do Rails esses métodos somem.

```
15.minutes
1.day.ago
2.weeks.from_now
Time.current      # respeita o fuso configurado na app; Time.now não
```

E o par que você vai usar mais que qualquer outro:

```
nil.blank?      # true
"".blank?       # true
"  ".blank?     # true  ← só espaço também conta como vazio
[].blank?       # true
0.blank?        # false  ← zero NÃO é vazio

"oi".present?   # true  ← present? é o contrário de blank?
```

Pegadinha para quem vem de Python: em Ruby puro, só `nil` e `false` são falsos. `if 0` executa. `if ""` executa. É justamente por isso que `blank?` existe.

3. Migrations

Uma migration é uma **mudança no banco, versionada em código**. Ela roda uma vez, em ordem, em todas as máquinas: a sua, a do colega e o servidor.

```
bin/rails generate migration CreateUsers
```

Isso cria `db/migrate/20260803142949_create_users.rb`. O número é a data e hora — é ele que define a ordem.

```
class CreateUsers < ActiveRecord::Migration[8.1]
  def change
    create_table :users do |t|
      t.string :name, null: false
      t.string :email, null: false
      t.string :password_digest, null: false
      t.string :course, null: false
      t.string :matricula, null: false
      t.datetime :terms_accepted_at, null: false

      t.timestamps
    end

    add_index :users, :email, unique: true
    add_index :users, :matricula, unique: true
  end
end
```

```
bin/rails db:migrate
```

Repare em duas coisas:

- `t.timestamps` cria `created_at` e `updated_at`, e o Rails mantém as duas sozinho.
- `add_index ... unique: true` é uma restrição no banco, não só validação no model. Isso importa: duas requisições simultâneas passam pela validação juntas (as duas consultam e as duas não acham ninguém), mas só uma vence o índice único. **Validação de model é para dar mensagem bonita; índice é para garantir.**

`schema.rb`

Depois de migrar, o Rails reescreve `db/schema.rb` — o retrato atual do banco. É ele que `db:prepare` usa para criar o banco de teste, e por isso **vai versionado**. Você nunca edita esse arquivo à mão.

Migrations são incrementais

Neste projeto a tabela `users` nasceu em três migrations, porque as funcionalidades chegaram em momentos diferentes:

```
20260803142949_create_users.rb          # o básico
20260803142950_add_email_verification_to_users.rb # confirmação de e-mail
20260803142951_add_password_reset_to_users.rb    # recuperação de senha
```

É assim que acontece na vida real. Nunca edite uma migration que já rodou em produção — crie outra.

4. Senha: o que nunca fazer

Nunca guarde senha em texto. Se o banco vazar — e bancos vazam — você entregou a senha de todos. E como as pessoas repetem senha, você entregou o e-mail e o banco delas junto.

Hash não é criptografia

Criptografia tem volta (com a chave). **Hash não tem volta:** é um caminho só. Você não guarda a senha; guarda o resultado de passar a senha pela função. Na hora do login, passa de novo e compara os resultados.

Por que bcrypt e não SHA-256

SHA-256 é rápido — **e isso é o problema.** Uma GPU testa bilhões de SHA-256 por segundo. bcrypt foi feito para ser **lento de propósito** e tem um fator de custo ajustável: quando o hardware melhora, você aumenta o custo.

bcrypt também gera um **salt** aleatório por senha. Sem salt, duas pessoas com a mesma senha teriam o mesmo hash, e uma tabela pronta (*rainbow table*) quebraria as duas de uma vez.

```
$2a$12$R9h/cIPz0gi.URNNX3kh20...
|  |  |  salt + hash
|  |  |  fator de custo (12 = 2^12 iterações)
|  |  |  versão do algoritmo
```

has_secure_password

```
# Gemfile
gem "bcrypt", "~> 3.1.7"
```

```
class User < ApplicationRecord
  has_secure_password validations: false
end
```

Isso dá ao model:

- `user.password = "..."` — recebe a senha em texto e grava o digest em `password_digest`
- `user.authenticate("...")` — devolve o usuário se bater, `false` se não

- `user.password_confirmation` — a confirmação, se você usar

A senha em texto **nunca toca o banco**. Ela vive na memória durante a requisição e some.

`validations: false` desliga as validações padrão porque a nossa política de senha é mais exigente. Ela mora em `PasswordPolicyValidator`.

5. Validações

```
validates :name, presence: true, length: { maximum: 120 }
validates :email, presence: true,
  uniqueness: { case_sensitive: false },
  format: { with: URI::MailTo::EMAIL_REGEXP }
validates :course, presence: true
validates :matricula, presence: true, uniqueness: true, format: { with: /\A\d{7}\z/ }
validates :terms_accepted_at, presence: true, on: :create
validate :password_meets_policy, if: -> { password.present? }
```

- `validates` (plural) usa um validador pronto. `validate` (singular) chama um método seu.
- `on: :create` só valida na criação — os termos são aceitos uma vez.
- `if:` recebe um lambda. Só valida a senha quando ela foi informada (na edição de perfil ela não é).

No console:

```
u = User.new(name: "Teste")
u.valid?           # false
u.errors.full_messages
```

Regra de ouro: normalize antes de validar

`Ana@UFOP.br` e `ana@ufop.br` são o mesmo e-mail. `20.112-34` e `2011234` são a mesma matrícula.

```
def self.normalize_email(email)
  email.to_s.strip.downcase
end

def self.normalize_matricula(matricula)
  matricula.to_s.gsub(/\D/, "")
end
```

Sem isso o índice único não serve para nada: o banco acha que são valores diferentes.

6. Associações

Uma tabela nunca basta. Todo sistema real tem tabelas que se referem umas às outras: usuário tem muitos pedidos; palestra acontece numa sala; sala tem muitas palestras.

Vamos criar a segunda tabela do projeto: `login_events`, o histórico de acessos da conta. É o que o seu banco usa para mandar "novo acesso detectado" por e-mail.

Quem guarda a chave

A relação um-para-muitos mora numa coluna só: a **chave estrangeira**. `login_events.user_id` aponta para `users.id`.

- quem **carrega** a coluna usa `belongs_to`;
- quem é **apontado** usa `has_many`.

Regra prática: a chave fica sempre no lado "muitos".

A migration

```
create_table :login_events do |t|
  t.references :user, null: false, foreign_key: true
  t.string :client, null: false
  t.string :ip_address
  t.datetime :occurred_at, null: false
  t.timestamps
end

add_index :login_events, [ :user_id, :occurred_at ]
```

- `t.references :user` cria a coluna `user_id`, o índice **e** a chave estrangeira.
- `foreign_key: true` faz o **banco** recusar uma linha órfã — não é só validação de model.
- O índice composto tem `user_id` primeiro porque a consulta é sempre "os últimos logins **deste** usuário". Num índice composto, **a ordem das colunas importa**: primeiro o que filtra.

Os dois lados

```
class User < ApplicationRecord
  has_many :login_events, dependent: :delete_all
end

class LoginEvent < ApplicationRecord
  belongs_to :user

  scope :recentes, -> { order(occurred_at: :desc) }
end
```

- `dependent: :delete_all` — apagar o usuário apaga o histórico junto. Sem isso sobram linhas apontando para um `id` que não existe mais.
- `belongs_to` já exige presença desde o Rails 5: `LoginEvent` sem `user` é inválido.
- `scope` é uma consulta com nome, e ela encadeia.

Usando

```
# Criar PELA associação já preenche o user_id — não precisa passar:
user.login_events.create!(client: "mobile", occurred_at: Time.current)

user.login_events.count
user.login_events.recentes.limit(5)
evento.user.name # navega para o outro lado
```

A armadilha N+1

```
users.each { |u| puts u.login_events.count }
```

Com 100 usuários isso são **101 consultas**: uma para buscar os usuários e cem para buscar o histórico de cada um. É o problema de performance número 1 de aplicação Rails.

A correção é uma palavra:

```
User.includes(:login_events).each { |u| puts u.login_events.count }
```

Duas consultas, sempre. O `seem-backend` usa a gem **Bullet**, que estoura um erro em desenvolvimento quando você escreve um N+1 sem perceber.

7. Concerns — o mixin da Aula 1, na prática

O `User` vai ganhar confirmação de e-mail e recuperação de senha. São dois assuntos que não conversam entre si. Jogar os dois no `user.rb` produz um arquivo de 800 linhas que ninguém abre com vontade.

Cada assunto vira um módulo em `app/models/concerns/`:

```
# app/models/concerns/email_verifiable.rb
module EmailVerifiable
  extend ActiveSupport::Concern

  CODE_TTL = 15.minutes
  RESEND_COOLDOWN = 1.minute

  included do
    scope :email_verified, -> { where.not(email_verified_at: nil) }
  end

  def email_verified?
    email_verified_at.present?
  end

  def issue_email_verification_code!
    code = format("%06d", SecureRandom.random_number(1_000_000))

    update!(
      email_verification_code_digest: BCrypt::Password.create(code),
      email_verification_expires_at: CODE_TTL.from_now,
      email_verification_sent_at: Time.current
    )

    code
  end
end
```

```
class User < ApplicationRecord
  include EmailVerifiable
  include PasswordResetable
end
```

- `extend ActiveSupport::Concern` habilita o bloco `included do ... end`, onde vão coisas que só fazem sentido na classe (scopes, validações, associações).
- O código completo dos dois concerns está em `app/models/concerns/`.

Três decisões dentro do concern, e o porquê de cada uma

1. O código de 6 dígitos também vira digest bcrypt. Ele é uma senha temporária. Quem lê o banco não pode confirmar a conta de ninguém.

2. O código em texto só existe no retorno do método. `issue_email_verification_code!` devolve o código para o service mandar por e-mail, e depois ele some. Não fica em coluna, nem em log.

3. `email_verified_at` é data, não booleano. Um booleano responde "sim". Uma data responde "sim, em 12 de agosto às 14h" — e isso você vai querer saber quando alguém abrir um chamado.

8. O console

```
bin/rails console
```

```
u = User.new(
  name: "Ana Souza",
  email: "ana@aluno.ufop.edu.br",
  password: "Automic@2026",
  course: "Engenharia de Controle e Automação",
  matricula: "2011234",
  terms_accepted_at: Time.current
)

u.valid?
u.save
u.password_digest      # o hash, não a senha
u.authenticate("Automic@2026") # devolve o user
u.authenticate("errada")      # false

User.count
User.where("email LIKE ?", "%ufop%")
User.find_by(email: "ana@aluno.ufop.edu.br")

codigo = u.issue_email_verification_code!
u.verify_email_code!(codigo)
u.email_verified?
```

`bin/rails console --sandbox` desfaz tudo ao sair. Bom para experimentar sem sujar o banco.

9. Seeds e fixtures

Seeds povoam o banco de desenvolvimento:

```
bin/rails db:seed
```

Fixtures são os dados dos testes, em `test/fixtures/users.yml`. O Rails carrega antes de cada teste e limpa depois — todo teste começa do mesmo estado.

```
ana:
  name: Ana Souza
  email: ana@aluno.ufop.edu.br
  password_digest: <%= BCrypt::Password.create("Automic@2026") %>
  email_verified_at: <%= 1.day.ago.to_fs(:db) %>
```

No teste, `users(:ana)` devolve esse registro.

10. Testando o model

```
test "guarda o digest e nunca a senha em texto" do
  user = build_user
  user.save!

  assert_not_equal "Automic@2026", user.password_digest
  assert user.authenticate("Automic@2026")
  assert_not user.authenticate("outra-senha")
end
```

```
bin/rails test
bin/rails test test/models/user_test.rb      # só um arquivo
bin/rails test test/models/user_test.rb:42   # só um teste
```

O Rails vem com **Minitest**. Se você conhece `pytest`, a ideia é a mesma; a sintaxe é `assert_*`.

11. Uma sutileza de segurança: `authenticate_by_email`

```
DUMMY_PASSWORD_DIGEST = BCrypt::Password.create("automic-timing-safe-dummy").freeze

def self.authenticate_by_email(email, password)
  user = find_by(email: normalize_email(email))
  digest = user&.password_digest || DUMMY_PASSWORD_DIGEST
  autenticado = BCrypt::Password.new(digest).is_password?(password.to_s)

  user if autenticado
end
```

Por que esse `DUMMY_PASSWORD_DIGEST`?

O jeito ingênuo seria `return nil unless user`. Só que bcrypt é lento **de propósito**. Se o e-mail não existe, a resposta volta em 1ms; se existe e a senha está errada, volta em 100ms. Cronometrando as respostas, dá para descobrir quem tem conta no sistema — é um **ataque de temporização**.

Rodando o bcrypt sempre, com um digest descartável quando não há usuário, as duas respostas custam o mesmo. Vamos usar esse método na Aula 3.

Exercício da aula

1. Crie as três migrations e rode `bin/rails db:migrate`.
2. Abra `db/schema.rb` e confira que as colunas estão lá.
3. Escreva o `User` com `has_secure_password` e as validações.

4. Crie os dois concerns e inclua no `User`.
5. No console: crie um usuário, veja o `password_digest`, teste `authenticate`.
6. Emita um código de verificação e confirme com ele. Depois tente confirmar de novo — tem que recusar.
7. Escreva os testes e rode `bin/rails test`.
8. Crie a migration de `login_events`, o model com `belongs_to` e o `has_many` no `User`.
9. No console: `users(:ana).login_events.create!(...)`, depois `user.login_events.recentes`.
10. Apague um usuário que tenha eventos e confira que o histórico foi junto.

Bônus: escreva o N+1 de propósito (`User.all.each { |u| puts u.login_events.count }`), olhe o log para contar as consultas, e conserte com `includes`.

Travou? `git checkout aula-02`.

Recapitulando

- Migration é a fonte da verdade do banco. O model não declara colunas.
- Índice único é garantia; validação é mensagem bonita. Você quer os dois.
- Senha vira hash bcrypt, com salt, e nunca volta.
- Concern é o mixin do Rails: um assunto por arquivo.
- Normalize antes de validar, ou o índice único não serve para nada.
- A chave estrangeira fica no lado "muitos": `belongs_to` carrega, `has_many` é apontado.
- Transação é tudo ou nada. Dentro dela, use os métodos com `!`.
- `includes` resolve N+1, e N+1 é o problema de performance mais comum em Rails.

Na próxima: as rotas. O usuário vai conseguir se cadastrar, entrar, sair e recuperar a senha.